Try running this code from bin/rails console:

```
Vehicle.includes(:vehicle_reservations).
    pluck('vehicles.name',
          'vehicle_reservations.starts_at',
          'vehicle_reservations.canceled')
```

.pluck() is even smart enough with invalid SQL, which requests fields that don't exist on a table, to figure out which tables to get the fields from.

The above query code can be rewritten as below and produce an equivalent result. This may be confusing to maintainers though, and a more the more explicit version above that selects the fields from the appropriate tables is clearer.

```
Vehicle.includes(:vehicle_reservations).
  pluck(:name, :starts_at, :canceled)
```

A major benefit of limiting the fields at query time is unlocking the possibility of Index Scans, and even Index Only Scans, to make retrieval very fast. You'll learn about those soon.

In the next section, you'll see how to save a round trip query by asking for fields to be returned before inserting data.

Save A SELECT With A RETURNING

PostgreSQL has a capability called the RETURNING keyword. It can be used to ask for fields to return before inserting data.

For fields that are populated by the database like a primary key field populated from a Sequence, you might write code to insert data and then query for the id value. Those second queries needed to look up the database supplied value can be replaced using the RETURNING keyword.

Try using the temp.users table you set up earlier. The table does not currently have a sequence associated with the id column so you'll set that up by running these statements from psql:

```
CREATE SEQUENCE temp_users_id_seq INCREMENT 1 START 1;

ALTER TABLE temp.users ALTER COLUMN id
SET DEFAULT nextval('temp_users_id_seq');
```

The first statement adds a sequence and the second statement assigns it to the id column. Now id will be populated by the sequence when you insert rows.

Try inserting a row, and add the RETURNING keyword asking for the id column.

```
INSERT into temp.users (name) VALUES ('bob') RETURNING id, name;
```